

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчёт о программном курсовом проекте на тему:

Улучшение СУБД ClickHouse

Выполнил студент:

4 курс

группы 2310

Шумакова Екатерина Сергеевна

Руководители проекта:

Каледин Максим Львович

доцент Департамента больших данных и информационного поиска
Факультета компьютерных наук НИУ ВШЭ

Миловидов Алексей Николаевич

сооснователь и технический директор ClickHouse Inc.

Москва, 2026

Содержание

Аннотация	4
Описание применения генеративной модели	5
Введение	6
Актуальность	6
Цель и постановка задачи	6
Объект и предмет исследования	7
Научная и практическая новизна	7
Практическая значимость	8
Структура работы	8
Обзор литературы и аналогов	9
ClickHouse и порядок выполнения агрегатов	9
Существующие time-series аналоги	9
Статистические аналоги	10
Объединяемые агрегаты и компактное состояние	10
Глава 1. Постановка задачи и контракт входных данных	11
1.1 Логический временной ряд.....	11
1.2 Что означает «точное» состояние	11
1.3 Общая поверхность API.....	11
1.4 Общие ошибки и ограничения	12
Глава 2. Архитектура объединяемого состояния и доказательство корректности.....	13
2.1 Представление состояния	13
2.2 Операция merge как объединение отсортированных последовательностей	13
2.3 Доказательство контракта	13
2.4 Независимость от порядка выполнения	14
2.5 Ограничения компактного prefix/suffix-состояния	14
2.6 Сложность	16
Глава 3. API и алгоритмы четырёх расширений	17

3.1 Общая позиционная модель	17
3.2 Лаговая линейная регрессия	17
3.3 Критерий ADF с фиксированным числом лагов	17
3.4 KPSS.....	18
3.5 Оценитель единственного сдвига среднего	18
3.6 Функции базового этапа	19
Глава 4. Реализация и численная устойчивость	20
4.1 Организация исходного дерева	20
4.2 Жизненный цикл состояния агрегата	20
4.3 Сериализация и защита от повреждённых состояний	20
4.4 Численная политика	21
4.5 Защитные проверки как часть реализации.....	21
Глава 5. Тестирование, эксперименты и оценка производительности	23
5.1 Стратегия тестирования.....	23
5.2 Сводный журнал приёмочных проверок.....	23
5.3 Покрытие API расширения.....	24
5.4 Независимый эксперимент на Python.....	24
5.5 Проверка эталонной модели на Python	25
5.6 Нативная оценка производительности	25
5.7 Размер состояния и производительность слияния	26
5.8 Дополнительные материалы базового этапа.....	26
5.9 Воспроизводимость.....	27
Заключение.....	28
Список использованных источников.....	30
Приложение А. Карта файлов проекта и материалов проверки	31
Приложение Б. Понятный порядок самостоятельной проверки.....	32
Приложение В. Краткий словарь терминов	34

Аннотация

Курсовой проект посвящён улучшению аналитических возможностей СУБД ClickHouse посредством разработки и локальной валидации четырёх статистических расширений для временных рядов: лаговой линейной регрессии, фиксированного статистического критерия Дики-Фуллера (ADF), статистики KPSS и оценителя точки изменения среднего. Вместе с тремя функциями базового этапа полный checkout содержит семь реализованных SQL-агрегатов для анализа временных рядов. Четыре новых расширения реализованы как обычные параметризованные агрегатные функции. Их общее состояние хранит все принятые пары (timestamp, value), канонизирует их сортировкой по timestamp и объединяет частичные состояния точным отсортированным слиянием. Такое решение не зависит от порядка поступления строк, блоков, шардов и формы дерева слияния, однако требует $O(n)$ памяти.

В отчёте формализованы контракт входных данных, merge-алгебра, positional semantics, ограничения по ресурсам и численная политика. Для ограниченного однодиапазонного prefix/suffix-состояния доказана незамкнутость при произвольном merge. Для усиленного контракта плотных соседних диапазонов реализован автономный C++-прототип с памятью $O(L)$; он прошёл по 1 764 проверки в оптимизированной и санитизированной сборках. Приведены сведения о реализации на C++, независимом Python-оракуле, SQL-фикстурах, GoogleTest, Distributed, AggregatingMergeTree и benchmark-запусках. В Release acceptance прошли 38/38 focused GoogleTest и 4/4 SQL-фикстуры. Независимый эксперимент с seed 20260915, $n=240$ и 120 повторами подтвердил восстановление коэффициентов AR и ожидаемое направление ADF/KPSS, а ошибка локализации mean shift составила 0,1083 наблюдения в среднем.

Ключевые слова: ClickHouse; временные ряды; агрегатные функции; merge-state; распределённая агрегация; ADF; KPSS.

Описание применения генеративной модели

При подготовке курсового проекта использовалась генеративная модель OpenAI GPT-5 в среде Codex (официальное описание модели: <https://openai.com/index/introducing-gpt-5/>). Модель применялась как вспомогательный инструмент при анализе структуры исходного репозитория, подготовке вариантов программной реализации, техническом редактировании программного кода и тестовых сценариев, структурировании материала отчёта, приведении текста к академическому стилю и оформлении итогового документа в форматах DOCX и PDF. Результаты работы модели рассматривались как рекомендации и черновые материалы и включались в проект только после проверки автором.

Автор самостоятельно определила окончательную цель и границы проекта. Вся математическая и теоретическая часть работы, включая выбор статистических конструкций, разработку формул, формулировку утверждений, построение контрпримеров, определение условий применимости методов и проверку математических выводов, была разработана и проверена автором лично.

Автор также лично выполнила все описанные в отчёте сборки и тестовые запуски, проверила журналы выполнения и соответствие полученных результатов установленным контрактам. Численные и экспериментальные результаты были самостоятельно проанализированы и интерпретированы автором. Окончательные инженерные решения, оценка ограничений реализованных методов и выводы курсового проекта принадлежат автору, которая несёт ответственность за содержание и достоверность представленной работы.

Введение

Актуальность

Современные аналитические системы обрабатывают временные ряды параллельно: данные читаются из нескольких частей, распределяются по блокам, агрегируются на шардах, сериализуются и затем объединяются на координаторе. Поэтому порядок, в котором строки физически попали в конкретный блок или были переданы функции `add`, не обязан совпадать с хронологическим порядком. Для коммутативных статистик, например суммы или количества, это почти незаметно. Для автокорреляции, лаговой регрессии, тестов стационарности и поиска разрыва среднего такое предположение принципиально меняет результат.

ClickHouse предоставляет мощный интерфейс пользовательской агрегатной функции с операциями добавления, слияния, сериализации и финализации. Однако сам интерфейс намеренно не обещает агрегату единого глобального порядка вызовов. Частичные состояния могут сливаться в разных бинарных деревьях; промежуточные состояния могут сохраняться в `AggregatingMergeTree` или передаваться между шардами. Если статистика зависит от соседних наблюдений, то простой однопроходный алгоритм, использующий порядок вызовов, не даёт воспроизводимого результата.

Практическая проблема состоит в том, чтобы одновременно получить математически определённую последовательность наблюдений, совместимость с обычной моделью ClickHouse aggregation и понятные ограничения ресурсов. Наивное решение - собрать весь ряд в массив и затем выполнить статистический расчёт - даёт корректную отправную точку, но само по себе не описывает поведение частичных aggregate states. Более сильное решение должно определить состояние, закон merge, serialization contract и правила ошибок. Именно эта задача лежит в центре проекта.

Цель и постановка задачи

Цель курсового проекта - разработать и проверить программное решение, направленное на расширение аналитических возможностей ClickHouse: четыре точные объединяемые агрегатные функции для статистического анализа временных рядов, а также формально описать общий контракт состояния, позволяющий использовать их в локальной, распределённой и сохраняемой агрегации.

Для достижения цели решаются следующие задачи:

1. исследовать особенности порядка выполнения агрегатов ClickHouse и существующие time-series аналоги;

2. задать типы входных ключей и значений, правила NULL, дубликатов, конечности и ограничения размера;
3. определить каноническое состояние, сериализацию и merge law;
4. доказать корректность sorted-union merge на допустимых состояниях;
5. проверить невозможность универсального компактного prefix/suffix state через контрпример;
6. реализовать четыре статистических finalizer-а и численные guard-ы;
7. проверить реализацию средствами C++ GoogleTest и SQL-функциональными тестами;
8. проверить двухшардовый Distributed путь и сохранение в AggregatingMergeTree;
9. сопоставить формулы с независимым Python-оракулом и измерить зависимость времени и размера состояния от параметров;
10. зафиксировать ограничения, открытые вопросы и границы применимости результатов.

Объект и предмет исследования

Объектом исследования является обработка и распределённая агрегация временных рядов в колоночной аналитической СУБД ClickHouse.

Предмет исследования - алгоритмы состояния aggregate function, порядок канонизации, операция merge, численная устойчивость и программные тесты для порядкозависимых статистик.

Научная и практическая новизна

Новизна проекта имеет инженерно-алгоритмический характер. Предложен единый exact keyed store-sort state, который сохраняет полную выборку и делает порядок наблюдений явной функцией ключа, а не порядка исполнения. Для этого состояния дано простое определение merge как канонического sorted union и явно отделена ассоциативность множества ключей от неизбежных ошибок дубликатов и переполнения cap.

Второй результат устанавливает точную границу применимости compact state. Для однодиапазонного состояния с ограниченным числом prefix/suffix элементов доказана незамкнутость при произвольной схеме слияния обычного aggregate. Одновременно для более сильного контракта - плотных соседних диапазонов и операции mergeAdjacent - дано конструктивное доказательство корректности и реализован автономный прототип с памятью $O(L)$. Такое разделение предотвращает небезопасную регистрацию компактного SQL-агрегата, но показывает, при каких гарантиях экономия памяти действительно достижима.

Третий результат - реализация четырёх расширений с намеренно узкими заявлениями. ADF и KPSS возвращают статистики без p-value; mean shift является описательным one-break

estimator; лаговая регрессия использует fixed order. Это снижает риск приписать прототипу статистические гарантии, которые не следуют из реализованных соглашений.

Практическая значимость

Результат может использоваться как основа для экспериментального анализа временных рядов непосредственно в ClickHouse, без выгрузки каждой группы в отдельное приложение. Состояние поддерживает partial aggregation, -State/-Merge, шардовую агрегацию и AggregatingMergeTree. Параметры cap и work budget превращают неконтролируемый расход памяти и времени в явные проверяемые условия. Набор evidence-пакетов и воспроизводимых команд позволяет использовать работу в учебном проекте по системному программированию, численным методам и базам данных.

Структура работы

В первой главе формализуются задача и входной контракт. Во второй рассматривается архитектура объединяемого состояния, приводится доказательство корректности и обосновываются ограничения компактного состояния. В третьей описываются четыре API и соответствующие статистические алгоритмы. Четвёртая глава посвящена реализации на C++, сериализации и численной устойчивости. В пятой представлены результаты тестирования, независимых экспериментов и оценки производительности. Далее приведены заключение, список источников и приложения.

Обзор литературы и аналогов

ClickHouse и порядок выполнения агрегатов

ClickHouse - распределённая колоночная СУБД, ориентированная на аналитические запросы [1]. Архитектура MergeTree предполагает существование физических data parts, которые создаются независимо, сортируются в соответствии с ключом таблицы и затем объединяются фоновыми процессами [2; 3]. Это полезно для чтения и индексации, но не означает, что группа данных представлена одним глобально упорядоченным потоком. ORDER BY задаёт физический порядок внутри соответствующих структур, но не является обещанием порядка вызовов пользовательской агрегатной функции на всех частях и шардах [2; 4].

Следствием является различие между семантическим ключом времени и execution order. В проекте timestamp используется только как скалярный ordering key. После сортировки наблюдения получают позиции $0, \dots, n-1$; численная разница между timestamp не используется как длительность, не заполняются пропуски и не оценивается календарная сезонность. Если необходимы статистические выводы для равномерной сетки по физическому времени, пользователь должен выполнить resampling до вызова агрегата.

Существующие time-series аналоги

ClickHouse уже содержит timeSeriesGroupArray, который группирует пары времени и значения, сортирует их и имеет собственную семантику обработки дубликатов [5; 6]. Этот агрегат является важным архитектурным прецедентом: state принадлежит одной SQL-группе, массив элементов может накапливаться с отложенной сортировкой, а merge может использовать линейное слияние упорядоченных диапазонов. Но он не является тестом стационарности, AR-регрессией или детектором точки изменения.

arrayAutocorrelation и аналогичные функции позволяют анализировать уже построенный массив, но не задают самостоятельный mergeable state для timestamp-ключей [7]. Поэтому проект не пытается переименовать существующую array-функцию, а исследует более общий contract: обычный агрегат принимает пары (timestamp, value), сохраняет canonical state и может участвовать в Distributed/persisted aggregation.

Внутренний перечень возможных задач анализа временных рядов в ClickHouse включает проверку стационарности, поиск точек структурного разрыва и прогнозирование [8]. В рамках курсового проекта область исследования намеренно ограничена четырьмя функциями с фиксированными параметрами и контролируруемыми вычислительными затратами. Такой подход соответствует инженерному требованию сначала зафиксировать API и набор подтверждающих материалов, а затем обсуждать возможное включение изменений в основную ветвь проекта.

Статистические аналоги

Лаговая авторегрессия восходит к классической схеме представления наблюдения через предыдущие значения [9]. ADF строится на регрессии первых разностей на запаздывающее значение уровня и лаги разностей [10; 11]. KPSS проверяет стационарность вокруг уровня или детерминированного тренда, используя накопленные остатки и long-run variance с Bartlett/Newey-West weights [12; 13]. Welford предложил устойчивое накопление моментов [14], а Chan, Golub и LeVeque исследовали обновление и pairwise combination дисперсий [15; 16]. Эти результаты использованы как основания для численной политики mean-shift и общих моментов.

Важна граница интерпретации. Статистика ADF сама по себе не является p-value и требует таблиц критических значений или response surface. KPSS также требует выбранной асимптотической/конечной выборочной конвенции, а bandwidth влияет на результат. Поэтому API в проекте возвращает только явно реализованные величины и не скрывает отсутствующие статистические соглашения за автоматически сгенерированным p-value.

Объединяемые агрегаты и компактное состояние

В распределённых системах агрегатное состояние должно поддерживать локальное накопление и объединение частичных результатов. Идеи distributive и algebraic aggregation описаны в литературе по аналитическим операторам [17], а параллельное объединение моментов - в работах [15; 16; 18]. Но порядкозависимый функционал может требовать информации о внутренних соседях, которая не выражается фиксированным числом сумм.

External sorting показывает, что упорядоченные runs можно безопасно объединять, если сохраняется их полное содержимое и определённое правило merge [19; 20]. Это соответствует выбранному exact state, но противоречит попытке хранить только границы диапазона. Таким образом, проект занимает консервативную позицию: сначала полный state с доказуемым merge, затем - при наличии специального планировщика и строгого adjacency contract - возможная оптимизация.

Глава 1. Постановка задачи и контракт входных данных

1.1 Логический временной ряд

Для одной SQL-группы допустимый логический ряд определяется формулой (1).

$$X = ((t_0, x_0), \dots, (t_{n-1}, x_{n-1})), \quad t_0 < t_1 < \dots < t_{n-1} \quad (1)$$

Каждое значение x_i конечное в арифметике Float64, timestamp уникален, а число строк не превосходит заданного `max_samples`. Входная последовательность может приходить в любом порядке. Сортировка по timestamp выполняется перед любой операцией, которая зависит от позиции.

Timestamp поддерживает UInt32, UInt64, DateTime и DateTime64. Числовые значения integer и floating-point преобразуются в Float64; Decimal текущим factory не принимается. Строки с NULL аргументом пропускаются стандартной nullable-логикой ClickHouse. Это означает, что NULL способен изменить число и расстояния positional observations; данное поведение должно быть явно указано в документации.

1.2 Что означает «точное» состояние

Слово «точное» относится к сохранению всех принятых конечных keyed samples и к отсутствию потери строк при допустимом merge. Оно не означает точную вещественную арифметику и не устраняет округление Float64. При переносе на другую платформу допускаются различия в последних разрядах, поэтому acceptance сравнивает результаты с абсолютными и относительными допусками, отдельно проверяя NaN, Inf, bounds и deterministic tie behaviour.

Изменение параметров после создания функции запрещено. При merge проверяются function kind, постоянные параметры, serialization version и max_samples. Нельзя интерпретировать байты состояния одной функции как состояние другой. Пустое состояние является identity. Результат недоопределённого finalizer-a - NaN в соответствующих полях, а не молчаливое нулевое значение.

1.3 Общая поверхность API

Полный состав реализованного семейства и формы возвращаемых значений приведены в Таблице 1. Семейство создавалось в два этапа: сначала были реализованы три базовые диагностики, затем к ним добавлены четыре новых статистических расширения.

Таблица 1 - Семь реализованных функций: три базовые и четыре новых

Функция	Параметры	Результат
timeSeriesAutocorrelation	lag[, max_samples]	Float64
timeSeriesLjungBoxTest	max_lag[, model_df[, max_samples]]	(statistic Float64, p_value Float64)
timeSeriesDurbinWatson	[max_samples]	Float64
timeSeriesLaggedLinearRegression	order[, max_samples]	(intercept Float64, coefficients Array(Float64))
timeSeriesADFStatistic	augmentation_lags[, deterministic[, max_samples]]	(statistic Float64, coefficient Float64, observations UInt64)
timeSeriesKPSSTest	regression[, bandwidth[, max_samples]]	(statistic Float64, bandwidth UInt64, observations UInt64)
timeSeriesMeanShiftChangePoint	min_segment[, max_samples]	(split_index UInt64, score Float64, mean_before Float64, mean_after Float64, sse Float64)

Итого в checkout зарегистрированы семь SQL aggregate-функций. К базовому этапу относятся timeSeriesAutocorrelation, timeSeriesLjungBoxTest и timeSeriesDurbinWatson. Новыми расширениями, разработанными на этапе продолжения курсового проекта, являются timeSeriesLaggedLinearRegression, timeSeriesADFStatistic, timeSeriesKPSSTest и timeSeriesMeanShiftChangePoint. Такое разделение объясняет, почему в цели указаны четыре новых расширения, а в Таблице 1 перечислены семь реализованных функций.

Все функции являются private-preview. Основной gate - enable_time_series_aggregate_functions = 1; factory также сохраняет legacy compatibility gate enable_time_series_table.

1.4 Общие ошибки и ограничения

Duplicate timestamp - hard error. Duplicate не усредняется, не выбирается по arrival order и не разрешается в пользу одного из значений. Hefinite value (NaN, +Inf, -Inf) отклоняется. max_samples положителен, по умолчанию равен 1 000 000 и не может превышать 10 000 000. Превышение cap - ошибка, а не усечение.

Валидация выполняется на нескольких границах: при добавлении, канонизации, merge, serialization и deserialization. Это важно для Distributed и AggregatingMergeTree: нарушение контракта в одном partial state не должно превращаться в недействительный итоговый результат.

Глава 2. Архитектура объединяемого состояния и доказательство корректности

2.1 Представление состояния

Shared state содержит vector записей {timestamp, value} и флаг sorted/dirty. В упорядоченном входном потоке новая запись добавляется амортизированно за $O(1)$. Если timestamp не больше предыдущего arrival key, vector помечается dirty. При merge или finalization dirty state сортируется по timestamp, проверяется строгая уникальность и становится canonical.

Сериализованный extension envelope содержит идентификатор функции, фиксированные параметры, версию формата и затем versioned payload keyed samples. Десериализатор не должен заранее резервировать неограниченный объём по недостоверному count: count, версия, порядок ключей, finiteness и cap проверяются до принятия состояния.

2.2 Операция merge как объединение отсортированных последовательностей

Пусть $C(S)$ - уникальное представление множества пар с timestamp в строго возрастающем порядке. Для непересекающихся наборов ключей операция и её основные алгебраические свойства задаются формулой (2).

$$\begin{aligned} S \oplus T &= C(S \cup T), \\ (S \oplus T) \oplus U &= S \oplus (T \oplus U), \quad S \oplus T = T \oplus S. \end{aligned} \quad (2)$$

Sorted union реализуется двумя указателями. Если текущий ключ левого состояния меньше текущего ключа правого, копируется левая запись; если больше - правая. Равенство приводит к ошибке. После исчерпания одного массива копируется хвост другого. Для canonical inputs время и временная память равны $O(n_1+n_2)$. Если вход dirty, сначала требуется сортировка $O(n \log n)$.

2.3 Доказательство контракта

Рассмотрим три допустимых состояния S , T , U , чьи множества timestamp попарно не пересекаются. Операция set union ассоциативна и коммутативна, что зафиксировано во второй строке формулы (2). Каноническая сортировка одного и того же конечного множества даёт единственный результат. Поэтому

то есть $(S \oplus T) \oplus U = S \oplus (T \oplus U)$. Аналогично $S \oplus T = T \oplus S$. Пустое состояние не меняет множество и является identity.

Следует подчеркнуть область применимости доказательства. Если ключи пересекаются, merge не является функцией разрешения конфликта, а выдаёт ошибку. Если сумма размеров

превышает cap, операция также завершается ошибкой. Поэтому корректность относится к valid disjoint-key inputs; она не утверждает, что любое произвольное повреждённое дерево merge должно завершаться успешным числом.

2.4 Независимость от порядка выполнения

Канонический вектор зависит только от множества допустимых пар, поэтому результат не зависит от:

- порядка строк в исходном запросе;
- разбиения на блоки;
- порядка частей MergeTree;
- interleaving данных на шардах;
- parenthesization бинарного merge tree;
- сериализации и обратной десериализации canonical state.

Finalizer получает одну и ту же positional sequence. Последние биты Float64 могут зависеть от компилятора, CPU и режима округления, поэтому численный acceptance формулируется через допуски, а не универсальное требование bitwise identity.

2.5 Ограничения компактного prefix/suffix-состояния

Интуитивно можно попытаться хранить только диапазон timestamp, несколько первых и последних значений и лаговые суммы. Далее отрицательный и положительный результаты формулируются отдельно, чтобы не смешивать обычный контракт ClickHouse aggregate со специальным контрактом упорядоченных диапазонов.

Утверждение 1 (незамкнутость однодиапазонного компактного состояния). Рассмотрим семейство состояний, каждое из которых описывает один плотный диапазон, хранит фиксированномерные числовые сводки и не более L первых и последних наблюдений. Если его единственная точная композиция - конкатенация соседних диапазонов, то это семейство не замкнуто относительно произвольного дерева merge, разрешённого обычной агрегатной функции.

Доказательство. Пусть корректный итоговый ряд состоит из singleton states с ключами 1, 2, 3 и значениями v_1 , v_2 , v_3 . Reducer имеет право сначала выбрать состояния 1 и 3. Они не соседние. Если операция отклонит такое слияние, допустимый набор данных может завершиться ошибкой только из-за выбранной формы дерева. Если же два фрагмента будут свёрнуты во внешний envelope [1, 3], состояние ошибочно объявит его плотным либо должно будет отдельно сохранить дыру. После прихода ключа 2 его уже нельзя вставить операцией конкатенации: он находится внутри envelope.

Лаговая сумма произведений, используемая в контрпримере, записана в формуле (3).

$$T(X) = \sum_{i=1}^{n-1} x_{i-1} x_i \quad (3)$$

Для полного ряда точный вклад первого лага равен $v_1v_2+v_2v_3$; состояния 1 и 3 не содержат ни одного из этих переходов. Контрпример масштабируется: reducer может сначала собрать нечётные ключи, а чётные предоставить позже. Точное продолжение потребует хранить произвольное число разрозненных subranges или внутренние наблюдения, то есть $\Omega(n)$ данных в худшем случае. Следовательно, ограниченное однодиапазонное состояние не замкнуто относительно произвольного дерева merge. Утверждение не запрещает все возможные алгоритмы сжатия и не относится к приближённым методам; оно ограничено указанным классом состояний и обычным aggregate-контрактом.

Утверждение 2 (корректность при adjacent merge). Пусть A и B - плотные соседние диапазоны одной серии с одинаковой конфигурацией. Состояние хранит общие центрированные моменты, отдельные сводки пар для каждого лага $h \leq L$, первые и последние $\min(L, n)$ значений и необходимые prefix-статистики KPSS. Тогда $\text{mergeAdjacent}(S(A), S(B))$ даёт то же математическое состояние, что и прямой проход по конкатенации $A \parallel B$.

Для доказательства достаточно разбить пары каждого лага на три непересекающиеся группы: внутри A, через границу и внутри B. Первая и третья группы уже учтены в частичных состояниях. Каждая граничная пара целиком находится в suffix A и prefix B, поскольку её расстояние не превышает L. Поэтому все граничные вклады восстанавливаются точно. Индукция по дереву, в котором каждое слияние соединяет соседние диапазоны, доказывает эквивалентность прямому упорядоченному проходу. В точной арифметике равенство строгое; для Float64 используется численный допуск. Объём состояния равен $O(L)$.

В проекте этот положительный результат доведён до автономного C++-прототипа: реализованы `appendSuccessor`, `mergeAdjacent`, проверка `gaps/overlaps/duplicates`, сериализация и вычисление ACF, Ljung-Box, Durbin-Watson, AR(1) и KPSS. Прототип не является зарегистрированным SQL aggregate, оператором планировщика или частью `Distributed/AggregatingMergeTree`. Для такой интеграции потребовался бы отдельный `planner/operator contract`, сохраняющий плотность и соседство диапазонов после `spill`, `retry`, `remote aggregation` и `persistence`.

Поток данных от физических строк к глобальному финализатору показан на Рисунке 1.

Канонический timestamp-порядок формируется внутри агрегата



Результат не зависит от порядка строк, границ блоков, шардов и формы дерева слияния

Рисунок 1 - Поток данных и канонизация состояния агрегата

2.6 Сложность

Асимптотические оценки основных операций сведены в Таблицу 2.

Таблица 2 - Асимптотическая сложность операций и финализаторов

Операция или функция	Время после сортировки	Дополнительная память
add	амортизированно $O(1)$	$O(1)$ на запись
dirty canonicalization	$O(n \log n)$	$O(n)$
canonical merge	$O(n_1+n_2)$	$O(n_1+n_2)$ временно
autocorrelation	$O(n)$	$O(1)$
Ljung-Box	$O(nH)$	$O(1)$
Durbin-Watson	$O(n)$	$O(1)$
lagged regression	$O(rp^2)$	$O(p^2)$
ADF	$O(rc^2)$	$O(c^2)$
KPSS	$O(nq)$	$O(1)$
mean shift	$O(n)$	$O(n)$ transient suffix moments

Здесь p - порядок регрессии, q - KPSS bandwidth, $r=n-p$ для AR, а c - число QR columns для ADF. State size и serialized payload всегда остаются $O(n)$.

Глава 3. API и алгоритмы четырёх расширений

3.1 Общая позиционная модель

После сортировки timestamp заменяется канонической позицией. Лаг единицы означает следующую запись в отсортированном векторе, независимо от фактической разницы времени. Неравномерные timestamp разрешены, но не превращаются в эквидистантные физические интервалы. Это компромисс между определённым merge-контрактом и универсальностью временных шкал.

3.2 Лаговая линейная регрессия

Для fixed order p строятся строки дизайна $t=p, \dots, n-1$, а модель задаётся формулой (4).

$$y_t = \alpha + \sum_{j=1}^p \phi_j y_{t-j} + \varepsilon_t \quad (4)$$

Коэффициенты выдаются в порядке $(\phi_1, \dots, \phi_p)$. В отличие от попытки сливать локальные коэффициенты, state merge-ит только наблюдения; дизайн и fit строятся один раз после canonicalization. Это необходимо, поскольку среднее локальных моделей в общем случае не равно модели на объединённом лаговом дизайне.

Для устойчивости response и predictor columns центрируются и масштабируются. Затем streaming Givens QR выполняет решение небольшой системы без column pivoting. Проверяются положительные residual degrees of freedom, rank, non-finite intermediate values, scaled reciprocal condition estimate и work budget $r \cdot p^2 \leq 100000000$. При неуспехе возвращается tuple фиксированной формы с NaN-полями.

3.3 Критерий ADF с фиксированным числом лагов

ADF использует ту же каноническую позицию, но регрессирует первую разность на лаг уровня и лаги разностей согласно формуле (5).

$$\Delta y_t = d_t + \gamma y_{t-1} + \sum_{j=1}^p \psi_j \Delta y_{t-j} + \varepsilon_t \quad (5)$$

Поддерживаются none, constant и trend. В trend mode линейный тренд - это номер канонической строки, а не численное значение timestamp. Возвращаются coefficient γ , t-ratio и

observations=n-p-1. При недостатке данных observations сохраняется информативным, а statistic/coefficient могут быть NaN.

Admission guard согласован с fixed-lag guard в statsmodels.adfuller [21] и приведён в формуле (6).

$$p \leq \left\lfloor \frac{n}{2} \right\rfloor - d - 1 \quad (6)$$

где d - число deterministic terms. Дополнительно проверяются residual degrees of freedom, rank, conditioning, backward-error resolution floor и $r^2 \leq 100000000$. Автоматического выбора лага и p-value в API нет.

3.4 KPSS

Для level из ряда вычитается sample mean, а для trend оцениваются intercept и positional linear trend. Остатки e_t , cumulative residual path S_t , sample autocovariance и итоговая статистика определяются формулой (7).

$$\begin{aligned} S_t &= \sum_{i=0}^t e_i, & \hat{\gamma}_h &= \frac{1}{n} \sum_{t=h}^{n-1} e_t e_{t-h}, \\ \hat{\omega}_q^2 &= \hat{\gamma}_0 + 2 \sum_{h=1}^q \left(1 - \frac{h}{q+1} \right) \hat{\gamma}_h, \\ \text{KPSS} &= \frac{n^{-2} \sum_{t=0}^{n-1} S_t^2}{\hat{\omega}_q^2}. \end{aligned} \quad (7)$$

Default q вычисляется собственной floor-конвенцией проекта. Explicit q неотрицателен, ограничен 1024 и должен быть меньше max_samples; defined statistic требует $q < n$. При non-positive/non-finite long-run variance, неразрешимой вариации или $nq > 100000000$ statistic возвращается как NaN. Bandwidth и observations могут оставаться заполненными.

3.5 Оцениватель единственного сдвига среднего

Для каждого split k, удовлетворяющего $\text{min_segment} \leq k \leq n - \text{min_segment}$, оценивается сумма внутрисегментных квадратов отклонений и описательный score по формуле (8).

$$\text{SSE}(k) = \sum_{i < k} (x_i - \bar{x}_{0:k})^2 + \sum_{i \geq k} (x_i - \bar{x}_{k:n})^2, \quad (8)$$

$$\text{score} = \max\left(0, 1 - \frac{\text{SSE}(k^*)}{\text{SSE}_0}\right).$$

Выбирается k^* , минимизирующий SSE. При общем SSE SSE_0 формула (8) нормирует улучшение в безразмерный score.

Native finalizer сначала range-scales values, затем поддерживает prefix Welford state и напрямую накопленные suffix Welford states. Subtractive reconstruction suffix SSE из total и prefix была отклонена: при сильном разрыве она может привести к катастрофической компенсации.

Для устойчивого выбора tied candidates вводится $\gamma_n = n\epsilon / (1 - n\epsilon)$, где ϵ - binary64 machine epsilon. Более поздний кандидат принимается только если improvement превышает $8\gamma_n \max(|\text{candidate}|, |\text{incumbent}|)$. Поэтому накопленное округление не создаёт искусственный change point на длинном no-improvement series, а численно равные candidates сохраняют earliest split.

3.6 Функции базового этапа

Три функции, реализованные на базовом этапе проекта, используют то же состояние. ACF рассчитывается относительно overall mean и при lag zero равна 1 для non-constant series. Статистика и асимптотическая вероятность Ljung-Box заданы формулой (9).

$$Q(H) = n(n+2) \sum_{h=1}^H \frac{\rho_h^2}{n-h}, \quad p = \Pr\{\chi_{H-d}^2 \geq Q(H)\} \quad (9)$$

Это единственная функция семейства, возвращающая p-value, и то в рамках её асимптотических предположений. Durbin-Watson использует отношение суммы квадратов соседних разностей к сумме квадратов уровней. Пустые, короткие, постоянные или недопустимые случаи дают NaN согласно конкретному контракту.

Глава 4. Реализация и численная устойчивость

4.1 Организация исходного дерева

Extension state и factory находятся в `src/AggregateFunctions/TimeSeries/AggregateFunctionTimeSeriesStatisticalExtensions.h/.cpp`; глобальная registration call добавлена в `src/AggregateFunctions/registerAggregateFunctions.cpp`. Для функций базового этапа используются соседние файлы `AggregateFunctionTimeSeriesDiagnostics.h/.cpp`, а их native-тесты находятся в `gtest_time_series_diagnostics.cpp`. Тесты четырёх новых расширений находятся в `src/AggregateFunctions/tests/gtest_time_series_statistical_extensions.cpp`.

В `coursework`-пакете исходники `extension` намеренно не дублируются: `authoritative production changes` находятся в `checkout src` и `tests`, а пакет содержит `research`, копию `baseline implementation` и `evidence`. Это важно учитывать при описании результата: `package` является `evidence bundle`, а не вторым `source tree`.

Полный исходный код и материалы воспроизводимой проверки доступны в репозитории проекта: <https://github.com/shumakovaes/ClickHouse/tree/coursework/time-series-extensions/coursework> (дата обращения: 18.09.2026).

4.2 Жизненный цикл состояния агрегата

1. `Factory` проверяет число и типы аргументов, `preview setting` и константные параметры.
2. `add` преобразует значение в `Float64`, проверяет `finiteness` и `cap`, сохраняет `timestamp/value`.
3. `Out-of-order arrival` помечает `vector dirty`.
4. Перед `serialization`, `merge` и `finalization` `vector` сортируется и проверяется на строгую уникальность.
5. `merge` сопоставляет `envelope` и выполняет `two-pointer union`.
6. `insertResultInto` выбирает `finalizer` по `Kind` и выдаёт типизированный `tuple`.

Для всех четырёх расширений фиксированные параметры сохраняются в `envelope`. Это предотвращает ситуацию, при которой `-Merge` объединяет состояния с разными `order`, `deterministic mode`, `bandwidth` или `min segment`.

4.3 Сериализация и защита от повреждённых состояний

Сериализация `canonical state` обеспечивает одинаковый порядок байтов для одного и того же `keyed dataset`. В `envelope` записываются версия формата, `function kind`, параметры и `delegated`

payload. Deserializer проверяет unsupported version/kind, truncation, malformed count, non-increasing keys, non-finite values, несовместимый cap и mismatch параметров.

Низкоуровневый GoogleTest позволяет передавать произвольные повреждённые bytes и проверять, что ошибка возникает безопасно. SQL-тест не может самостоятельно изготовить любой opaque state, поэтому malformed envelope tests остаются ответственностью native harness.

4.4 Численная политика

ACF и Ljung-Box используют midpoint/range transform и compensated sums. Durbin-Watson масштабирует значения относительно maximum absolute value. Regression columns центрируются и масштабируются перед QR. KPSS и mean shift вычисляются в scaled coordinates, а результат переводится к заявленной размерности.

Эти преобразования уменьшают avoidable cancellation, но не восстанавливают информацию, потерянную при преобразовании входа в Float64. Undefined result является частью API contract: он отображается NaN в scalar или tuple field. Для mean shift положительное SSE может переполниться в +Inf, при этом split и dimensionless score остаются валидными; слишком малое SSE может underflow в zero.

4.5 Защитные проверки как часть реализации

Система guard-ов выполняет две функции. Во-первых, она предотвращает слишком дорогой финализатор при больших p , q или n . Во-вторых, она отделяет обычную статистическую неопределённость от ложного finite результата, полученного из численной ошибки. В частности, residual-resolution floor ADF не разрешает округлению превратить почти нулевую residual variance в огромный искусственный t-ratio.

Guard не является доказательством универсальной статистической мощности. Его следует трактовать как policy проекта: при небезопасном для заявленной точности вычислении система возвращает NaN и сохраняет фиксированную форму результата.

Измеренная зависимость размера state от числа наблюдений показана на Рисунке 2.

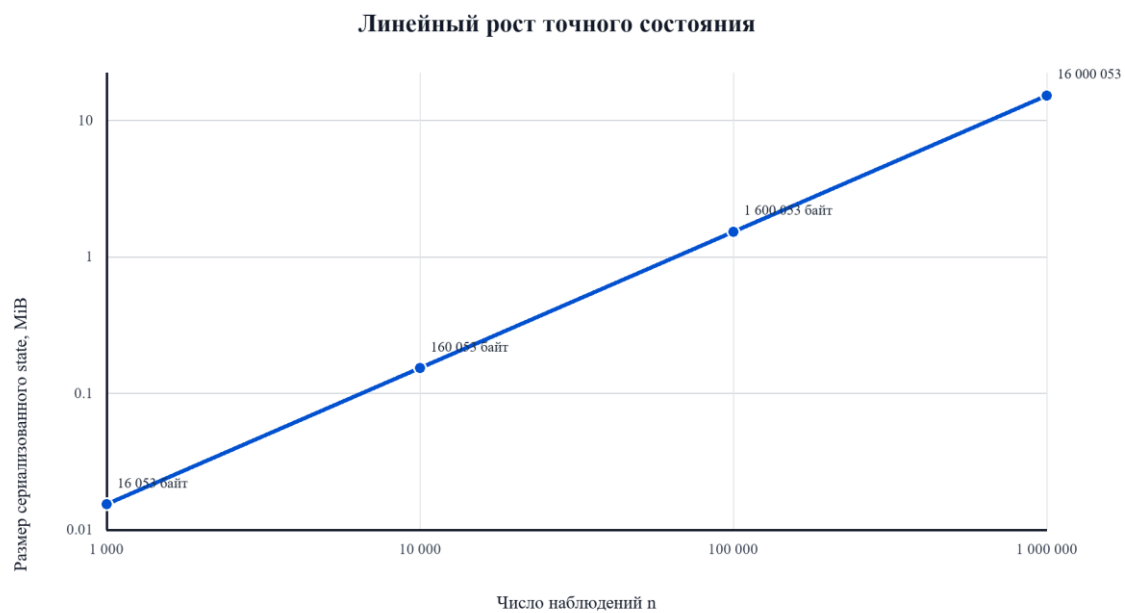


Рисунок 2 - Измеренный рост размера сериализованного состояния

Глава 5. Тестирование, эксперименты и оценка производительности

5.1 Стратегия тестирования

Тестовая стратегия разделяет уровни:

- unit-level state tests проверяют add, sort, merge, serialization и errors;
- finalizer tests проверяют формулы, типы tuple, NaN и numerical boundaries;
- direct SQL fixture проверяет factory dispatch, setting, NULL, Decimal rejection и state combinators;
- Distributed fixture проверяет serialized partial merge и cross-shard duplicate propagation;
- AggregatingMergeTree fixture проверяет persistence, merge частей и duplicate-state failure;
- Python oracle проверяет математические формулы независимо от C++;
- benchmarks измеряют время, bytes и merge variants отдельно.

Это предотвращает смешение разных типов доказательств. Наличие source file само по себе не доказывает, что factory вызывается из SQL, а Python formula test не доказывает правильность native serialization.

5.2 Сводный журнал приёмочных проверок

Сводные результаты локальной приёмочной проверки приведены в Таблице 3.

Таблица 3 - Сводный журнал локальной приемочной проверки

Проверка	Статус	Детали
Release full build	PASS	сохранённый лог 6838/6838
Release после изменения CMake	PASS	отдельная incremental verification 545/545, exit 0; не clean rebuild
Focused GoogleTest Release	PASS	38/38
Focused GoogleTest Debug	PASS	38/38
SQL 05161	PASS	1/1
SQL 05162	PASS	1/1
SQL 05163	PASS	1/1, Distributed
SQL 05164	PASS	1/1, AggregatingMergeTree
Выбранные новые примеры документации	PASS	7/7

Ключевая оговорка о сборке: 6838/6838 относится к сохранённому full Release build log. После изменения CMake зафиксирован отдельный incremental результат 545/545; его нельзя переименовывать в чистую полную пересборку. Все локальные native acceptance claims следует привязывать к соответствующим evidence-пакетам и binary identity.

5.3 Покрытие API расширения

05162 включает:

- preview gate, arity и parameter validation;
- representative UInt64/Float64 dispatch;
- NULL handling и Decimal rejection;
- exact tuple shapes и undefined cases;
- state combinators -State, -Merge, -MergeState;
- shuffled/interleaved rows;
- duplicate timestamps и non-finite values.

05161 проверяет три функции базового этапа. 05162 проверяет прямой SQL-контракт четырёх новых расширений. 05163 разделяет одну таблицу по двум localhost shards и проверяет direct Distributed finalization, serialized partial-state merge и duplicate errors для всех четырёх новых API. 05164 создаёт несколько AggregatingMergeTree parts, проверяет persistence/finalization и duplicate failure. Focused native suite содержит 23 теста расширений и 15 тестов базовых функций: всего $38 = 23 + 15$ тестов.

Оценка покрытия является targeted coverage assessment, а не LLVM/gcov percentage. Не утверждается, что проверены все scalar widths, все allocator failures, все QR threshold neighbourhoods или полный ClickHouse corpus.

5.4 Независимый эксперимент на Python

Независимый experiment runner использует seed 20260915, 120 повторов и 240 наблюдений. Он разделяет AR recovery, ADF direction, KPSS behavior, mean-shift localization и edge outcomes. Optional statsmodels использовался как cross-check, а не как основание для p-value claim.

Численные результаты эксперимента представлены в Таблице 4.

Таблица 4 - Результаты независимого Python-эксперимента

Сценарий	Численный результат
AR(1), $\phi=0.25$, noise SD=0.2	intercept bias 0.002854; ϕ RMSE 0.063852
AR(1), $\phi=0.70$, noise SD=1.0	intercept bias 0.009558; ϕ RMSE 0.051025
AR(2), $\phi=(0.50, -0.25)$, noise SD=0.5	ϕ_1 RMSE 0.062453; ϕ_2 RMSE 0.061536
ADF stationary/random walk	stationary более отрицателен в 120/120
KPSS level stationary/random walk	means 0.16800/0.86441; smaller в 113/120
KPSS detrended trend/level-only trend	means 0.07503/1.70315; smaller в 120/120
Mean shift	MAE 0.1083; exact 109/120; ± 12 - 120/120

Cross-check differences with optional statsmodels были порядка $5.33e-15$ для ADF, нулевыми для AR intercept, $1.67e-16$ и $3.05e-16$ для двух AR coefficients и $1.39e-17$ для KPSS. Это подтверждает согласие на выбранной fixture, но не универсальную битовую эквивалентность.

Отдельно проведён sensitivity experiment для функций базового этапа ACF и Ljung-Box с seed 20260916 и 200 повторами. В evidence сохранены 12 000 исходных строк для ACF и 1 800 строк для Ljung-Box, а также сводные таблицы, metadata и SHA-256. Этот запуск относится к базовым диагностикам и не смешивается с экспериментом четырёх новых расширений.

5.5 Проверка эталонной модели на Python

Независимый Python reference suite для четырёх расширений прошёл 25/25 тестов, а проверки самого extension experiment runner - 6/6. Они подтверждают формулы и обработку граничных случаев в независимой модели, но не заменяют native GoogleTest и SQL-проверки кода ClickHouse.

Отдельный benchmark измеряет только независимые batch finalizer-ы Python. Input generation и warm-up исключены из timed sample, tracemalloc отражает Python-traced allocation, а не RSS ClickHouse server. Mean-shift oracle намеренно $O(n^2)$, поэтому его нельзя сравнивать с native $O(n)$ finalizer как с равноправным throughput measurement.

При $n=4096$ медианное время составило 62,911 ms для AR $p=1$, 123,553 ms для AR $p=8$, 78,213 ms для ADF $p=0$, 130,798 ms для ADF $p=4$, 22,253 ms для KPSS $q=0$, 402,003 ms для KPSS $q=32$ и 5824,721 ms для Python mean shift.

5.6 Нативная оценка производительности

Native Release runner выполнил 92 timing rows на WSL2 host, с `max_threads=1`, одним warm-up и тремя повторениями. Размеры $n=1000$ и 10000 использовались для основной grid, а дополнительные boundary/scaling cases доходили до 100000 и 97657 наблюдений. Server VmRSS/VmHWM - process-wide snapshots, а wall time включает client/server protocol и deterministic expression evaluation.

До появления четырёх расширений для трёх базовых функций был сохранён отдельный Debug benchmark: 81 основное измерение, а также проверки размера состояния, fan-in и групп рядов. Его результаты не объединяются с 92 строками Release benchmark, поскольку режим сборки, состав функций и назначение запусков различаются.

Ключевая проверка work cap для KPSS: при $n=97656$, $q=1024$ произведение равно 99 999 744 и вычисление допускается; при $n=97657$, $q=1024$ произведение равно 100 000 768 и statistic

становится NaN. Это показывает, что guard является наблюдаемой частью контракта, а не только недостижимой веткой исходного кода.

5.7 Размер состояния и производительность слияния

В Release state/merge benchmark использовались $n=\{1000, 10000, 100000, 1000000\}$, число partial states $\{1, 4, 16, 64\}$ и три повторения. Измерялись direct finalization, raw serialized bytes, MergeState и final Merge.

Размеры сериализованного состояния приведены в Таблице 5.

Таблица 5 - Размер сериализованного состояния

n	Serialized state, 1 partial state
1 000	16 053 bytes
10 000	160 053 bytes
100 000	1 600 053 bytes
1 000 000	16 000 053 bytes

Размер подтверждает $O(n)$ storage. Extension parameters меняют небольшой envelope overhead, но не превращают состояние в $O(p)$ или $O(q)$: все timestamp/value samples сохраняются.

Для $n=100000$ и типичных parameters median wall times сведены в Таблицу 6.

Таблица 6 - Время слияния и финализации при $n = 100\,000$

Extension	MergeState	Merge c finalization
lagged regression, $p=4$	0,09-0,10 s	0,12-0,14 s
ADF, $p=2$	0,09-0,11 s	0,12-0,13 s
KPSS, $q=8$	0,09-0,11 s	0,10-0,12 s
mean shift	0,09-0,10 s	0,10-0,11 s

Benchmark даёт сравнительную реализационную информацию на одном host, но не обещает производительность production-кластера, конкретный RSS allocator или SLA.

5.8 Дополнительные материалы базового этапа

Архив базового этапа сохранён отдельно от итоговой приёмки расширений. Для трёх исходных диагностик lean aggregate target завершил 5 672/5 672 действий, а unified ClickHouse target - 1 167/1 167. На той версии были получены 15/15 focused GoogleTest и 1/1 numbered SQL test, включая двухшардовое слияние и сохранение в AggregatingMergeTree. Независимая NumPy/SciPy/statsmodels-модель прошла 15/15 тестов, а baseline experiment suite - 6/6.

Дополнительный изолированный C++-прототип реализует усиленный ordered-range contract: новый ключ можно добавить только как непосредственного преемника, а два непустых

состояния можно слить только при соседстве их плотных диапазонов. Он хранит $O(L)$ данных и вычисляет ACF, Ljung-Box, Durbin-Watson, AR(1) и KPSS; тесты включают случайные допустимые merge trees, ошибки gaps/overlaps/duplicates и round-trip сериализации. Повторный запуск исходного файла standalone.cpp с SHA-256 5c88f938471530c5bec76a91452186da82bcf574f183fcf1b6db05a80dae52cb компилятором Ubuntu Clang 21.1.8 подтвердил по 1 764 успешные проверки в оптимизированной сборке и в отдельной сборке с ASan/UBSan без обнаруженных ошибок. Прототип служит сравнительным доказательным артефактом и намеренно не считается частью native SQL API. Его результаты не складываются с итоговыми 38/38 GoogleTest и 4/4 SQL fixtures: эти наборы проверяют разные реализации и контракты.

5.9 Воспроизводимость

Для native acceptance необходимо использовать checkout, содержащий extension implementation, registry, gtest и fixtures 05161-05164, затем выполнить Release/Debug commands из coursework/REPRODUCING.md. Для Python:

```
py -3 -m unittest discover -s reference/python -p "test_*.py"
py -3 evidence/experiments/run_extension_experiments.py \
--seed 20260915 --n 240 --reps 120 --adf-lags 1 \
--change-min-segment 20 --output-dir <new-output-directory>
```

Финальный report PDF и SHA-256 evidence являются артефактами конкретного run. При переносе workspace следует использовать package-relative paths и новый operator-selected output directory, не подменяя новый запуск старым ledger.

Заключение

В полном checkout курсового проекта реализованы и проверены семь SQL aggregate-функций для временных рядов: три диагностики базового этапа и четыре новых статистических расширения. Центральным результатом этапа продолжения стали `timeSeriesLaggedLinearRegression`, `timeSeriesADFStatistic`, `timeSeriesKPSSTest` и `timeSeriesMeanShiftChangePoint`. Все семь функций используют общее keyed store-sort состояние, которое сохраняет конечные выборки (`timestamp`, `value`), приводит их к уникальному каноническому порядку и объединяет частичные состояния двухуказательной операцией `sorted union`.

Формальное свойство `merge` следует из ассоциативности и коммутативности `set union` и единственности канонической сортировки. Оно действует на допустимых состояниях с непересекающимися `timestamp` и не маскирует ошибки дубликатов или `cap overflow`. Благодаря этому итог не зависит от `arrival order`, `block boundaries`, `shard placement` и `merge-tree shape`.

Для ограниченного однодиапазонного `prefix/suffix state` формально доказана незамкнутость при произвольном дереве `merge`: `schedule 1, 3, 2` требует две внутренние границы, которые нельзя восстановить из результата раннего несоседнего слияния, а хранение всех нерешённых фрагментов в худшем случае требует $\Omega(n)$ памяти. Вместе с тем доказано, что при гарантии плотных соседних диапазонов сводки внутренних и граничных лаговых пар объединяются точно. Этот усиленный контракт реализован в автономном $O(L)$ C++-прототипе и проверен в двух режимах по 1 764 тестовых условий; он не выдаётся за обычный SQL aggregate или готовый `planner operator`.

Реализованные API имеют узкие и проверяемые контракты. Лаговая регрессия использует `fixed order` и `guarded Givens QR`. ADF возвращает `fixed-lag t-ratio` и `coefficient` без `p-value`. KPSS использует собственную `Bartlett bandwidth convention` и также не возвращает `p-value`. Mean shift является описательным `one-break estimator` с $O(n)$ native scan и устойчивой обработкой `suffix moments`.

Локальная валидация подтвердила 38/38 focused GoogleTest в Release и Debug, где 23 теста относятся к четырём расширениям, а 15 - к трём базовым функциям. Успешно выполнены 4/4 SQL fixtures, двухшардовая Distributed агрегация, `AggregatingMergeTree` persistence и 7/7 выбранных новых примеров документации. Независимый Python reference suite для четырёх расширений прошёл 25/25 тестов, а extension experiment suite - 6/6; эти проверки не заменяют native validation. Независимый experiment показал ожидаемое направление ADF/KPSS и среднюю ошибку mean-shift localization 0,1083 наблюдения. State-size benchmark подтвердил линейный рост serialized payload.

Ограничения не скрываются: состояние требует $O(n)$ памяти, positional model не заменяет resampling, ADF/KPSS не имеют p-value, mean shift не является calibrated test, a native performance измерена на одном WSL2 host. Сохранённый full Release лог 6838/6838 и отдельная incremental verification 545/545 после изменения CMake должны оставаться разными фактами; последняя не является clean rebuild. Таким образом, работа предоставляет воспроизводимый локально проверенный coursework prototype, но не утверждает включение функций в официальный upstream ClickHouse release.

Список использованных источников

- [1] Schulze S. et al. ClickHouse: Lightning Fast Analytics for Everyone. *Proceedings of the VLDB Endowment*. 2024. Vol. 17, No. 12. P. 3731-3744. DOI: 10.14778/3685800.3685802.
- [2] ClickHouse. MergeTree Table Engine [Электронный ресурс]. 2026. URL: <https://clickhouse.com/docs/en/engines/table-engines/mergetree-family/mergetree> (дата обращения: 10.09.2026).
- [3] ClickHouse. Data Parts [Электронный ресурс]. 2026. URL: <https://clickhouse.com/docs/managing-data/core-concepts/parts> (дата обращения: 10.09.2026).
- [4] ClickHouse. Choosing a Primary Key [Электронный ресурс]. 2026. URL: <https://clickhouse.com/docs/guides/best-practices/sparse-primary-indexes> (дата обращения: 10.09.2026).
- [5] ClickHouse. timeSeriesGroupArray [Электронный ресурс]. 2026. URL: <https://clickhouse.com/docs/sql-reference/aggregate-functions/reference/timeSeriesGroupArray> (дата обращения: 10.09.2026).
- [6] ClickHouse. AggregateFunctionTimeSeriesGroupArray.h [Электронный ресурс]. GitHub. URL: <https://github.com/ClickHouse/ClickHouse/blob/master/src/AggregateFunctions/TimeSeries/AggregateFunctionTimeSeriesGroupArray.h> (дата обращения: 10.09.2026).
- [7] ClickHouse. ClickHouse 26.4 Release Presentation [Электронный ресурс]. URL: <https://presentations.clickhouse.com/2026-release-26.4/> (дата обращения: 10.09.2026).
- [8] ClickHouse. Intern Tasks 2025/2026, Issue 87836 [Электронный ресурс]. GitHub. URL: <https://github.com/ClickHouse/ClickHouse/issues/87836> (дата обращения: 10.09.2026).
- [9] Yule G. U. On a Method of Investigating Periodicities in Disturbed Series, with Special Reference to Wolfer's Sunspot Numbers. *Philosophical Transactions of the Royal Society A*. 1927. Vol. 226. P. 267-298.
- [10] Dickey D. A., Fuller W. A. Distribution of the Estimators for Autoregressive Time Series with a Unit Root. *Journal of the American Statistical Association*. 1979. Vol. 74. P. 427-431. DOI: 10.1080/01621459.1979.10482531.
- [11] Said S. E., Dickey D. A. Testing for Unit Roots in Autoregressive-Moving Average Models of Unknown Order. *Biometrika*. 1984. Vol. 71, No. 3. P. 599-607.
- [12] Kwiatkowski D., Phillips P. C. B., Schmidt P., Shin Y. Testing the Null Hypothesis of Stationarity Against the Alternative of a Unit Root. *Journal of Econometrics*. 1992. Vol. 54. P. 159-178.
- [13] Newey W. K., West K. D. A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica*. 1987. Vol. 55. P. 703-708.
- [14] Welford B. P. Note on a Method for Calculating Corrected Sums of Squares and Products. *Technometrics*. 1962. Vol. 4, No. 3. P. 419-420.
- [15] Chan T. F., Golub G. H., LeVeque R. J. Updating Formulae and a Pairwise Algorithm for Computing Sample Variances. In: *COMPSTAT 1982*. P. 30-41.
- [16] Chan T. F., Golub G. H., LeVeque R. J. Algorithms for Computing the Sample Variance: Analysis and Recommendations. *The American Statistician*. 1983. Vol. 37. P. 242-247.
- [17] Gray J. et al. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. *Data Mining and Knowledge Discovery*. 1997. Vol. 1. P. 29-53.
- [18] Pébay P. Formulas for Robust, One-Pass Parallel Computation of Covariances and Arbitrary-Order Statistical Moments. Sandia National Laboratories Report SAND2008-6212. 2008.
- [19] Larson P.-Å., Graefe G. Memory Management During Run Generation in External Sorting. In: *Proceedings of SIGMOD*. 1998. P. 472-483.
- [20] Larson P.-Å. External Sorting: Run Formation Revisited. *IEEE Transactions on Knowledge and Data Engineering*. 2003. Vol. 15, No. 4. P. 961-972.
- [21] statsmodels developers. Source Code for statsmodels.tsa.stattools.adfuller [Электронный ресурс]. 2026. URL: https://www.statsmodels.org/stable/_modules/statsmodels/tsa/stattools/_stattools.html (дата обращения: 15.09.2026).

Приложение А. Карта файлов проекта и материалов проверки

Основные файлы проекта и материалы проверки приведены в Таблице 7. Знак * в пути означает набор файлов или каталог конкретного запуска; полные имена и контрольные суммы сохранены в coursework/MANIFEST.md и соответствующих SHA-256 manifest-файлах.

Таблица 7 - Основные файлы проекта и материалы проверки

Файл или папка	Что подтверждает
coursework/{README.md,REPRODUCING.md,MANIFEST.md}	Состав пакета, статус проверок и команды повторного запуска
coursework/research/	Проектирование состояния, правила слияния, ограничения и оценка покрытия
coursework/research/COMPACT_STATE_CLOSURE_PROPOSITION.md	Формальные утверждения о незамкнутости обычного compact merge и корректности adjacent merge
coursework/comparison/standalone_cpp/standalone.cpp	Автономная реализация ordered-range состояния с памятью $O(L)$
src/AggregateFunctions/TimeSeries/AggregateFunctionTimeSeriesDiagnostics.{h,cpp}	Код трёх функций базового этапа
src/AggregateFunctions/TimeSeries/AggregateFunctionTimeSeriesStatisticalExtensions.{h,cpp}	Код общего состояния и четырёх новых финализаторов
src/AggregateFunctions/tests/gtest_time_series_*.cpp	15 базовых и 23 extension-теста, всего 38
tests/queries/0_stateless/05161_*.sql-05164_*.sql	Базовый, прямой, Distributed и AggregatingMergeTree SQL-сценарии
docs/.../timeSeries*.mdx	Семь страниц пользовательской документации с проверяемыми примерами
coursework/reference/python/	Независимые Python-формулы и тесты базовых и новых функций
coursework/evidence/release-build-*/	Полная Release-сборка и отдельная incremental-проверка
coursework/evidence/{native-acceptance-*,debug-gtest-*/}	Release/Debug логи 38/38, SQL 4/4, конфигурация и SHA-256
coursework/evidence/docs-examples-*/	Результат проверки семи примеров документации
coursework/evidence/{native-benchmark-*,state-merge-benchmark-*/}	92 Release timing rows, размер состояния и время слияния
coursework/evidence/benchmarks/extensions-20260916-final/	Python benchmark четырёх расширений
coursework/evidence/experiments/extension_results_*/	Независимый эксперимент для четырёх расширений
coursework/evidence/experiments/baseline_sensitivity_*/	Sensitivity experiment для ACF и Ljung-Box
coursework/evidence/{trusted-reference,standalone-validation}/	Проверка эталонной модели и изолированного C++-прототипа

Приложение Б. Понятный порядок самостоятельной проверки

Это приложение объясняет проверку проекта человеку, который раньше не собирал ClickHouse. **Сборка** - это превращение исходного кода в готовую программу. **Тест** - автоматическая проверка ожидаемого поведения. **Лог** - текстовый файл, в котором сохранён ход команды и её результат. Каждый новый запуск нужно сохранять отдельно, чтобы новые данные не смешались со старыми.

1. **Записать, какая версия кода проверяется.** Выполнить `git rev-parse HEAD`, записать название ветки и результат `git status`. Полученная длинная строка однозначно обозначает версию исходного кода. Также нужно убедиться, что в checkout присутствуют файлы семи функций, их регистрация и тесты.
2. **Проверить инструменты и собрать программу.** Убедиться, что CMake и компилятор доступны, затем выполнить Release-сборку в отдельном чистом каталоге. Release - это оптимизированный вариант программы, близкий к обычному использованию. Чистый каталог нужен, чтобы результат не зависел от старых промежуточных файлов. Для повторения всей проверки отдельно собирается Debug-вариант, удобный для поиска ошибок.
3. **Сохранить полный ход сборки.** Сохранить обычные сообщения и сообщения об ошибках, то есть `stdout` и `stderr`. В журнале явно написать, была ли сборка чистой (`clean`) или использовала ранее созданные файлы (`incremental`). Эти два вида сборки нельзя выдавать друг за друга.
4. **Запустить C++-тесты.** Запустить `focused GoogleTest` только для рассматриваемых функций. Сохранить команду, код завершения и полный лог. Ожидаемый итог для проверенного checkout - 38 тестов: 15 для функций базового этапа и 23 для четырёх новых расширений.
5. **Запустить четыре SQL-сценария.** Выполнить 05161, 05162, 05163 и 05164 по отдельности. Первый проверяет три базовые функции, второй - прямую работу четырёх новых функций, третий - работу на двух частях распределённой системы, четвёртый - сохранение и последующее объединение промежуточных состояний.
6. **Проверить распределённую работу понятным способом.** В сценарии 05163 данные делятся между двумя шардами, то есть двумя частями условного кластера, а ClickHouse должен получить тот же общий результат. В сценарии 05164 промежуточные результаты сначала сохраняются в таблице `AggregatingMergeTree`, а затем объединяются. Также проверяется, что повторяющийся `timestamp` распознаётся как ошибка.

7. **Сравнить с независимыми формулами Python.** Запустить reference-тесты и эксперимент с seed 20260915. Seed - это фиксированное начальное число генератора случайных данных; оно позволяет повторить тот же набор примеров. Результаты следует записать в новый каталог.
8. **Измерить скорость и размер состояния.** Запустить native benchmark и отдельную проверку размера/слияния состояния. Все эти запуски должны использовать один и тот же исполняемый файл ClickHouse; его путь и SHA-256 нужно записать вместе с результатами.
9. **Отдельно проверить ordered-range прототип.** Скомпилировать standalone.cpp сначала с оптимизацией, затем с ASan/UBSan. Оба запуска должны сообщить PASS: 1764 checks. Эти проверки подтверждают компактное состояние только при слиянии соседних плотных диапазонов; они не означают, что создан обычный ClickHouse aggregate.
10. **Проверить целостность файлов.** Сверить SHA-256 сохранённых логов и результатов с manifest-файлами. SHA-256 можно понимать как цифровой отпечаток: если файл изменился хотя бы на один байт, его отпечаток обычно станет другим.
11. **Честно записать окончательный статус.** PASS означает, что конкретная локальная команда завершилась успешно и дала ожидаемый результат. Если какая-либо локальная команда не запускалась или завершилась с ошибкой, это следует указать прямо и не заменять отметкой PASS.

При повторной проверке нельзя переносить старые отметки PASS и старые счётчики без чтения новых логов. Значение 545/545, полученное после изменения CMake, является incremental-проверкой, а не полной чистой Release-сборкой.

Приложение В. Краткий словарь терминов

Canonical state - состояние, в котором записи отсортированы по timestamp и timestamp уникальны. **Partial state** - состояние, построенное на части строк или шарде. **Finalizer** - алгоритм, превращающий canonical state в scalar или tuple result. **Positional semantics** - модель, где соседство определяется позицией после сортировки, а не физической разницей timestamp. **Work guard** - проверяемый предел вычислительной работы, при превышении которого возвращается undefined result. **Private preview** - функция checkout-a, доступная только при специальном setting и не являющаяся утверждённой частью официального upstream release.